

Project Portfolio: Machine Learning for Insurance

About the Project

This project simulates real-world machine learning applications for a fictional insurance company called Protect Your Tomorrow. The goal is to solve common business problems using supervised machine learning techniques, including classification, regression, and sensitive data protection.

Main Objectives

1. Find similar customers based on their characteristics.
2. Predict whether a customer will receive insurance payments (binary classification).
3. Estimate how many payments they are likely to receive (regression).
4. Apply data obfuscation to protect personal information without degrading model performance.

This project could easily be extended to a real insurance company for:

- Personalized marketing (targeting similar customers)
- Risk scoring for claim approval
- Forecasting claim frequency
- Privacy-preserving analytics in compliance with GDPR/LGPD

Data Exploration and Cleaning

Loading and preparing data

```
df = pd.read_csv('/datasets/insurance_us.csv')  
  
df = df.rename(columns={'Gender': 'gender', 'Age': 'age', 'Salary': 'income', 'Family members': 'family_members',  
                        'Insurance benefits': 'insurance_benefits'})  
  
df['age'] = df['age'].astype(int)  
  
df['insurance_benefits_received'] = df['insurance_benefits'].apply(lambda x: 1 if x > 0 else 0)
```

Task 1: Similar Customers (kNN)

```
from sklearn.neighbors import NearestNeighbors  
  
from sklearn.preprocessing import MaxAbsScaler  
  
feature_names = ['gender', 'age', 'income', 'family_members']
```

```
def get_knn(df, n, k, metric):
    nbrs = NearestNeighbors(n_neighbors=k, metric=metric)
    nbrs.fit(df[feature_names])
    distances, indices = nbrs.kneighbors([df.iloc[n][feature_names]])
    return df.iloc[indices[0]].assign(distance=distances[0])
```

Task 2: Classification (Insurance Payment Prediction)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import f1_score, confusion_matrix
```

```
for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    print(f"k={k}, F1={f1_score(y_test, y_pred):.2f}")
```

Task 3: Linear Regression (Number of Payments)

```
class MyLinearRegression:
    def __init__(self):
        self.weights = None

    def fit(self, X, y):
        X2 = np.append(np.ones([len(X), 1]), X, axis=1)
        self.weights = np.linalg.inv(X2.T @ X2) @ X2.T @ y

    def predict(self, X):
        X2 = np.append(np.ones([len(X), 1]), X, axis=1)
        return X2 @ self.weights
```

Task 4: Data Obfuscation

```
# Obfuscation
P = np.random.default_rng(seed=42).random((X.shape[1], X.shape[1]))
```

```
while np.linalg.det(P) == 0:
    P = np.random.default_rng().random((X.shape[1], X.shape[1]))
X_obf = X @ P
X_recovered = X_obf @ np.linalg.inv(P)
```

Task 5: Linear Regression with Obfuscated Data

```
# Regression using obfuscated data
model_obf = MyLinearRegression()
model_obf.fit(X_train @ P, y_train)
y_pred_obf = model_obf.predict(X_test @ P)
```

General Conclusion

- Preprocessing quality was crucial to the success of the models.
- Simple models (like kNN and Linear Regression) perform very well when data is properly prepared.
- Obfuscation with invertible matrices is an effective technique for protecting sensitive data without degrading model performance.
- The project demonstrates a complete pipeline: from exploratory analysis to data security, applicable to real-world scenarios in companies handling confidential information.